

Diseño de Compiladores I – Compiladores e Interpretes

Trabajo Práctico N° 2 - 2026

Fecha de entrega: 18-09-2026

OBJETIVO

Construir un Parser (Analizador Sintáctico) que reciba los tokens detectados por el Analizador Léxico creado en el Trabajo Práctico N° 1, y compruebe si el código fuente se corresponde con la sintaxis de un lenguaje con las siguientes características:

SINTAXIS GENERAL:

Nota: En la descripción de los temas generales y particulares puede aparecer el agregado o eliminación de algún símbolo / palabra reservada durante el Análisis Léxico.

Programa:

- Programa constituido por un nombre de programa, seguido de un bloque de sentencias declarativas, y a continuación, un bloque de sentencias ejecutables.
- El bloque de sentencias ejecutables estará delimitado por **BEGIN** y **END**.
- Cada sentencia debe terminar con punto y coma ";".
- El programa comenzará con un nombre, seguido por un conjunto de sentencias delimitado por 'begin' y 'end'.

Ejemplo de programa:

```
programa_1
...    // Sentencias declarativas
begin
...    // Sentencias ejecutables
end
```

Sentencias declarativas:

- Sentencias de declaración de datos para los tipos de datos correspondientes a cada grupo según la consigna del Trabajo Práctico 1, con la siguiente sintaxis:

<tipo> <lista_de_variables>;

Donde <tipo> puede ser (Según tipos correspondientes a cada grupo):

- **shortint**
- **ushortint**
- **integer**
- **utinteger**
- **longint**
- **ulongint**
- **singlef**
- **doublef**

Las variables de la lista se separan con coma (",")

- Incluir declaración de funciones, con la siguiente sintaxis:

<tipo> **FUNCTION** ID (<parámetros_formales>)

<sentencias_declarativas_de_la_funcion> // conjunto de sentencias declarativas

BEGIN

<sentencias_ejecutables_de_la_funcion> // conjunto de sentencias ejecutables

END

Donde:

- <parámetros_formales> será una lista de identificadores precedidos por su tipo y separados por ','; Entonces, cada parámetro tendrá la siguiente estructura:

<tipo> ID

Las funciones deben tener, al menos, un parámetro.

- <sentencias_declarativas_de_la_funcion> es un conjunto de sentencias declarativas, incluyendo declaración de otras funciones.
- <sentencias_ejecutables_de_la_funcion> es un conjunto de sentencias ejecutables, incluyendo sentencias de retorno, que pueden aparecer en cualquier lugar del cuerpo de sentencias ejecutables de la función, con la siguiente estructura:

ret (<retorno>;

<retorno> podrá ser cualquier expresión aritmética

Una función puede tener más de un retorno, y se admite la ausencia de retorno (En el Trabajo Práctico 3 se indicará qué retorna una función sin retorno explícito)

Ejemplo:

```
Integer function f1 (integer x, doublef y)
  integer w ;
  integer function f11 (integer x1, integer x2)
  begin
    x2 := x1;
  end;
begin
  x := f11(x, 0$i);
  ...
  if (x > 1$i)
    ret (x + 2$i);
  else
    begin
      x := 2$i;
      ret (x * 2$i);
    end
  end_if;
end;
```

- Incorporar, como sentencia declarativa, la declaración de clases con la estructura que se muestra en los siguientes ejemplos:

```
CLASS ca
BEGIN
  INTEGER a;          // declaración de atributo
  INTEGER m(integer pa)
  BEGIN              // declaración de método
    ...
  END;
END;

CLASS cb
  SINGLEF b;  // declaración de atributo
  SINGLEF a;  // declaración de atributo
  SINGLEF n(INTEGER pb) // declaración de método
  BEGIN      // declaración de método
    ...
  END;
END;
```

Incorporar, como sentencia declarativa, la declaración de objetos de una clase determinada:

```
ca a1, a2;
cb b1, b2, b3
```

// **LÉXICO**: Incorporar, a la lista de palabras reservadas, la palabra **CLASS**

Sentencias ejecutables:

- Asignaciones donde el lado izquierdo puede ser un identificador, y el lado derecho una expresión aritmética. Los operandos de las expresiones aritméticas pueden ser variables, constantes, invocaciones a función o método, referencias a atributos u otras expresiones aritméticas.

No se debe permitir anidamiento de expresiones con paréntesis.

- Las invocaciones a función tendrán el siguiente formato:

ID(<parametros_reales>)

Cada parámetro real puede ser una expresión aritmética, variable o constante:

Ejemplos de asignaciones:

```
a := b * 2$i;      z := f1(a, b) + j;      w := a / f3(a+b, 10$i) + f4(1.2);
```

- Las referencias a atributos o métodos de un objeto se escribirán utilizando el '.' como se indica en los ejemplos:

```
a1.a = 3_i;
b1.b = 1.2;
```

```

b2.b = b1.a;
x := b1.m(1$1);
y := b1.n(2$1) + w;

```

(todos los chequeos semánticos para estas referencias, se efectuarán en las etapas siguientes)

- Cláusula de selección (**if**). Cada rama de la selección será un bloque de sentencias. La estructura de la selección será, entonces:

```
if (<condicion>) <bloque_de_sent_ejecutables> else <bloque_de_sent_ejecutables> end_if ;
```

El bloque para el **else** puede estar ausente.

La condición será una comparación entre expresiones aritméticas, variables, invocaciones a función o constantes, y debe escribirse entre “(“ ”)“.

El bloque de sentencias ejecutables puede estar constituido por una sola sentencia, o un conjunto de sentencias ejecutables delimitadas por **begin end**.

- Sentencia de salida de mensajes por pantalla. El formato será

```
pout(<cadena>);
```

```
o
```

```
pout(<expresion>);
```

Ejemplos:

```

pout('Hola mundo');           //Tema 9
pout( "Hola                    //Tema 10
      Mundo");
pout(a+ 5.0*b);               //Todos los temas

```

TEMAS PARTICULARES

Nota: La semántica de cada tema particular, se explicará y resolverá en las etapas 3 y 4 del trabajo práctico.

Temas 11 a 14: Sentencias de Control Iterativas

- **Tema 11:** *while repeat*

```
while ( <condicion> ) repeat <bloque_de_sentencias_ejecutables> ;
```

<condición> tendrá la misma definición que la condición de las sentencias de selección.

<bloque_de_sentencias_ejecutables> podrá contener una sentencia, o un grupo de sentencias ejecutables delimitadas por **begin end**.

// **Léxico:** Incorporar, a la lista de palabras reservadas, las palabras **WHILE** y **REPEAT**.

- **Tema 12:** *repeat until*

```
Repeat <bloque_de_sentencias_ejecutables> until ( <condicion> );
```

<condición> tendrá la misma definición que la condición de las sentencias de selección.

<bloque_de_sentencias_ejecutables> podrá contener una sentencia, o un grupo de sentencias ejecutables delimitadas por **begin end**.

// **Léxico:** Incorporar, a la lista de palabras reservadas, las palabras **REPEAT** y **UNTIL**.

- **Tema 13:** *repeat while*

```
repeat <bloque_de_sentencias_ejecutables> while ( <condicion> );
```

<condición> tendrá la misma definición que la condición de las sentencias de selección.

<bloque_de_sentencias_ejecutables> podrá contener una sentencia, o un grupo de sentencias ejecutables delimitadas por **begin end**.

Incorporar, a la lista de palabras reservadas, las palabras **REPEAT** y **while**.

- **Tema 14:**

```
From ID := <valor_inicial> to <valor_final> by <valor_actualización> repeat < bloque_de_sentencias_ejecutables > ;
```

<valor_inicial>, <valor_final> y <valor_actualización> serán constantes

<bloque_de_sentencias_ejecutables> podrá contener una sentencia, o un grupo de sentencias ejecutables delimitadas por **begin end**.

// **Léxico**: Incorporar, a la lista de palabras reservadas, las palabras **FROM**, **TO**, **BY** y **REPEAT**.

Temas 17 y 18: Asignaciones en expresiones

▪ **Tema 17: Asignaciones de expresiones en expresiones**

Se debe permitir asignar expresiones a los términos de expresiones aritméticas. Estas nuevas asignaciones, utilizarán el símbolo '=', y la expresión a asignar debe ir delimitada por paréntesis.

Nota: Se usarán paréntesis **solamente** para delimitar la expresión para este tipo de asignaciones, pero sin anidamiento

Ejemplos válidos:

```
a:=b=(2$1);          c := f(a=(3.0));      a:=b=(c+d)*e=(c/3$u1);  a:=b=(2$ui+d);
if (a=(3$ui)==3$ui) ...  if (a=(b) !=3$u1) ...    if (a=(b+c)==3.1) ...      a:=b+c=(4$s);
```

Ejemplos inválidos:

```
a:=b:=(2$1);          c := f(a:=(3.0));      a:=b:=(c+d)*e=(c/3$u1);  a:=b=(2$ui+d);
if (a:=(3$ui)==3$ui)  if (a=b!=3$u1) ...    if (a:=(b+c)==3.1) ...      a:=b+c:=(4$s);
...
```

▪ **Tema 18: Asignaciones de identificadores o constantes en expresiones**

Se debe permitir asignar identificadores o constantes a los términos de expresiones aritméticas. Estas nuevas asignaciones, utilizarán el símbolo '='.

Ejemplos válidos: (sólo a modo aclarativo, se subraya la asignación que se llevará a cabo en cada caso)

```
a:=b=2$1;              c := f(a=3.0);      a:=b=c+d*e=c/3$u1;  a:=b=2$ui+d;
if (a=3$ui==3$ui) ...  if (a=b!=3$u1) ...    if (a=b+c==3.1) ...      a:=b+c=4$s;
```

Ejemplos inválidos:

```
a:=b:=2$1;              c := f(a:=3.0);      a:=b:=c+d*e=c/3$u1;  a:=b=2$ui+d;
if (a:=3$ui==3$ui) ...  if (a:=b!=3$u1) ...    if (a:=b+c==3.1) ...      a:=b+c:=4$s;
```

Temas 19 a 21: Funciones con invocaciones y retornos con características especiales

▪ **Tema 19: Orden de evaluación de parámetros reales obligatorio (con asignaciones en expresiones)**

El programador **debe**, para cada invocación, indicar el orden en el cual quiere que se evalúe y se realice la asignación de los parámetros reales a los formales.

Las invocaciones tendrán la siguiente estructura:

```
ID ( <lista_parámetros_reales> ) [<lista_constantes>]
```

Donde:

<lista_parámetros_reales> será una lista de expresiones separadas por coma

<lista_constantes> será una lista de constantes separadas por coma

Por ejemplo:

```
a := f(x,y,z, ...) [1,2,3,...]
b := f2(a=1$i,b+3,5$i) [2,3,1]
```

// **Léxico**: Incorporar, a la lista de símbolos reconocidos, los corchetes '[' y ']'.

▪ **Tema 20: Orden de evaluación de parámetros reales opcional (con asignaciones en expresiones)**

El programador **puede**, para cada invocación, indicar el orden en el cual quiere que se evalúe y se realice la asignación de los parámetros reales a los formales.

Las invocaciones tendrán la siguiente estructura:

```
ID ( <lista_parámetros_reales> ) [<lista_constantes>] // [<lista_constantes>] es opcional
```

Donde:

<lista_parámetros_reales> será una lista de expresiones separadas por coma

<lista_constantes> será una lista de constantes separadas por coma

Por ejemplo:

```
a := f(x,y,z, ...) [1,2,3,...];
b := f2(a=1$1,b+3,5$1) [2,3,1];
c := f2(a,b,c);
```

Incorporar, a la lista de símbolos reconocidos, los corchetes '[' y ']'.
[1]

▪ **Tema 21: Funciones con retornos de tipos diferentes**

Se debe permitir inferir el tipo retornado por funciones que se definan con la siguiente estructura:

```
AUTO FUNCTION f(<lista_parámetros_formales>)
Begin
  <cuerpo_funcion>
End;
```

// **LÉXICO**: Incorporar, a la lista de palabras reservadas, la palabra **auto**.

Una función declarada con esta estructura debe tener, al menos, un retorno. Esta condición debe ser chequeada sintácticamente.

La semántica de esta funcionalidad se explicará en las etapas siguientes.

Temas 22 y 23: Declaraciones especiales

▪ **Tema 22: Variables en tiempo de compilación**

Permitir, dentro de las sentencias declarativas, la declaración de variables que sólo existen en el programa durante la compilación. Para ello, se utilizará la siguiente estructura:

```
comptime <tipo> <lista_variaables>;
```

// **LÉXICO**: Incorporar, a la lista de palabras reservadas, la palabra **comptime**.

La semántica de esta funcionalidad se explicará en las etapas siguientes.

▪ **Tema 23: Enumeraciones**

Incorporar la posibilidad de definir tipos que establezcan, en su definición, una lista de valores de alguno de los tipos básicos del lenguaje. La definición del tipo tendrá la siguiente estructura:

```
Typedef ID = [ <lista_valores> ];
```

Se debe permitir la declaración de variables del tipo definido por el usuario

Por ejemplo:

```
Typedef pares_ld = [2$1,4$1,6$1,8$1]; //define el tipo pares_ld
Pares_ld a, b, c; // Declara las variables a, b y c del tipo pares_ld
```

// **LÉXICO**: Incorporar, a la lista de palabras reservadas, la palabra **typedef**, y a la lista de símbolos aceptados, los corchetes..

Temas 24 al 27: Clases/Objetos

▪ **Tema 24: Asignación por estructura en objetos con código**

En la declaración de las clases se debe incluir, luego del nombre de la clase, un código que permitirá asignar objetos de clases diferentes, siempre que posean el mismo código (Esto será chequeado en la etapa siguiente)

Por lo tanto, la declaración de una clase, **para este tema** se realizará como se indica en el siguiente ejemplo:

```

CLASS ca c10
BEGIN
    INTEGER a;           // declaración de atributo
    INTEGER m(integer pa)
    BEGIN               // declaración de método
        ...
    END;
END;

CLASS cb c11
    SINGLEF b; // declaración de atributo
    SINGLEF a; // declaración de atributo
    SINGLEF n(INTEGER pb) // declaración de método
    BEGIN      // declaración de método
        ...
    END;
END;

```

▪ **Tema 25:** *Asignación por estructura en objetos sin código*

La declaración de las clases siguiendo la estructura indicada en la descripción de sentencias declarativas general:
La semántica de este tema se presentará en la etapa siguiente.

▪ **Tema 26:** *import/export por atributos y métodos*

En la declaración de las clases se puede incluir, luego de la declaración de un atributo o de un método, una lista de clases a las que ese atributo o método se exporta.

En la declaración de las clases, luego del nombre de la clase, se puede incluir una sentencia indicando de qué clase/s se importan atributos o métodos.

Por lo tanto, la declaración de una clase, **para este tema** se realizará como se indica en el siguiente ejemplo:

```

class A begin
    integer x1 export to B, C;
    singlef x2 export to C;
    integer m1(integer p) begin
        ...
    end; export to B;
end;

class B import from A begin
    integer x3; export to C;
    integer m2(A a1) begin
        a1.x1 := 1$i;
        a1.m1(1$i);
    end;
end;

class C import from A, B begin
    integer m3(A a1, B b2) begin
        a1.x2 := 2$i;
        b2.x3 := 3$i;
    end;
end;

```

// **LÉXICO:** Agregar a la lista de palabras reservadas, las palabras **import**, **from**, **export** y **to**.

▪ **Tema 27:** *Clases Amigas*

En la declaración de una clase se puede incluir, precediendo a la declaración de un atributo o de un método, la palabra **private**, y agregar, en el cuerpo de la clase, una lista de clases y funciones amigas, como se indica en el siguiente ejemplo:.

```

CLASS ca
BEGIN
    Private INTEGER a;           // declaración de atributo privado
    Private INTEGER m(integer pa)

```

```

BEGIN                                // declaración de método privado
...
END;
Friend cb;           // declara a la clase cb como amiga
Friend f;            // declara a la función f como amiga
END;

```

Esta funcionalidad se explicará en la etapa siguiente.

// **LÉXICO**: Agregar a la lista de palabras reservadas, las palabras **friend** y **private**;

Temas 28 y 29: Acceso a atributos

▪ **Tema 28: Acceso tradicional a atributos**

El acceso a un atributo de una clase se efectuará como se indicó en la declaración general para sentencias ejecutables. Es decir utilizando el punto.

Por ejemplo:

```

class ca begin           ca a1;
integer x;               a1.x = 3$i;
integer z;               a1.z = 3$i;
end;

```

▪ **Tema 29: Acceso posicional a atributos**

El acceso a un atributo de una clase, **para este tema**, se efectuará con una notación posicional.

Por ejemplo:

```

class ca begin           ca a1;
integer x;               a1[0$i] = 3$i;    // accede al atributo x
integer z;               a1[1$i] = 3$i;    // accede al atributo z
end;

```

Los índices pueden ser constantes o variables `a1[0$i]` o `a1[x]`

// **LÉXICO**: Agregar a la lista de símbolos reconocidos, los corchetes.

Temas 30 a 32: Herencia múltiple

Para los 3 temas:

En la declaración de una clase se puede incluir, en el cuerpo de la clase, una o más clases de las cuales esta clase hereda, mediante una sentencia `extend` seguida de una lista de nombres de clase, como se indica en el ejemplo:

```

class ca
begin
integer x;
integer z;
extends cb, cc, cd; // la clase ca hereda de cb, cc, cd
end;

```

Tema 30: Desambiguado Opción 1

Se explicará en la etapa 3

Tema 31: Desambiguado con prefijado

En el acceso a un atributo de una clase, ya sea tradicional o posicional, se debe permitir indicar el nombre de la clase a la que pertenece el atributo o método heredado.

Por ejemplo:

```

ca a1;
a1.x
a1.cb.x
a1.cc.x

o

a1[0$i]
a1.cb[1$i]

```

Tema 32: Desambiguado Opción 3

Se explicará en la etapa 3

Temas 33 a 35: Conversiones

■ Tema 33: Conversiones Explícitas de Entero a Punto Flotante

Se debe incorporar en todo lugar donde pueda aparecer una expresión, la posibilidad de utilizar la siguiente sintaxis:

```

tosf (<expresión>) // para grupos que tienen asignado el tema 7
todf (<expresión>) // para grupos que tienen asignado el tema 8

```

// **LÉXICO**: Incorporar a la lista de palabras reservadas, la palabra **tosf** o **todf** según corresponda.

■ Tema 34: Conversiones Explícitas de Punto Flotante a Entero

Se debe incorporar en todo lugar donde pueda aparecer una expresión, la posibilidad de utilizar la siguiente sintaxis:

```

tos (<expresión>) // para grupos que tienen asignado el tema 1
tous (<expresión>) // para grupos que tienen asignado el tema 2
toi (<expresión>) // para grupos que tienen asignado el tema 3
toui (<expresión>) // para grupos que tienen asignado el tema 4
tol (<expresión>) // para grupos que tienen asignado el tema 5
toul (<expresión>) // para grupos que tienen asignado el tema 6

```

// **LÉXICO**: Incorporar a la lista de palabras reservadas, la palabra **tos**, **tous**, **toi**, **toui**, **tol** o **toul** según corresponda.

■ Tema 35: Conversiones Implícitas

Se explicará y resolverá en trabajos prácticos 3 y 4.

SALIDA DEL COMPILADOR

El programa deberá leer un código fuente escrito en el lenguaje descripto, y deberá generar como salida:

- **Tokens detectados por el Analizador Léxico**
- **Estructuras sintácticas detectadas en el código fuente. Por ejemplo:**
 - Asignación
 - Sentencia **WHILE**
 - Sentencia **IF**
 - etc.(Indicando nro. de línea para cada estructura)
- **Errores léxicos y sintácticos presentes en el código fuente, indicando: nro. de línea y descripción del error. Por ejemplo:**
 - Línea 24: Error: Constante de tipo **integer** fuera del rango permitido.
 - Línea 43: Error: Falta paréntesis de cierre para la condición de la sentencia **IF**.
 - Línea 52: Warning: El identificador `identificadormuylargo` fue truncado a: `identificadormu`
- **Contenidos de la Tabla de símbolos**

CONSIDERACIONES GENERALES

- a) Utilizar **YACC** u otra herramienta similar para construir el Parser.
- b) Adaptar el Analizador Léxico del Trabajo Práctico 1 para convertirlo en el método o función ***integer yylex()*** (o el nombre que el Parser generado requiera). Tener en cuenta que el léxico deberá devolver al parser, en cada invocación, un token. Para los identificadores, constantes y cadenas, deberá devolver además, la referencia a la entrada de la Tabla de Símbolos donde se ha registrado dicho símbolo, utilizando ***yylval*** para hacerlo.
- c) Para aquellos tipos de datos que permitan valores negativos (**int**, **long**, **float**, **dfloat**) durante el Análisis Sintáctico se deberán detectar **constantes negativas**, modificando la tabla de símbolos según corresponda. Será necesario volver a controlar el rango de las constantes, ya que un valor aceptado para una constante por el Analizador Léxico, que desconoce su signo, podría estar fuera de rango si la constante es positiva.
 - Ejemplo: Las constantes de tipo **integer** pueden tomar valores desde -32768 a 32767. El Léxico aceptará la constante 32768 como válida, pero si se trata de una constante positiva, estará fuera de rango.
- d) Cuando se detecte un error, la compilación debe continuar.
- e) Conflictos: Eliminar **TODOS LOS CONFLICTOS SHIFT-REDUCE Y REDUCE-REDUCE** que se presenten al generar el Parser.

FORMA DE ENTREGA

Se deberá presentar:

- a) Código fuente completo y ejecutable, **incluyendo librerías del lenguaje** si fuera necesario para la ejecución
- b) Informe
 - Contenidos indicados en el enunciado del Trabajo Práctico 1
 - Descripción del proceso de desarrollo del Analizador Sintáctico: problemas surgidos (y soluciones adoptadas) en el proceso de construcción de la gramática, manejo de errores, solución de conflictos shift-reduce y reduce-reduce, etc.
 - Lista de no terminales usados en la gramática con una breve descripción para cada uno.
 - Lista de errores léxicos y sintácticos considerados por el compilador.
 - Conclusiones.
- c) Casos de prueba que contemplen **todas** las estructuras válidas del lenguaje. Incluir casos con errores sintácticos.